

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 2

Student Name: NISHA ANAND

Enrollment Number: 12023002022123

Department: CST

Institution: Institute of Engineering and Managent,Newtown

1. Module Name

Module Name: Unit Test Frameworks

2. Experiment Title

Integrated Test Automation Using Unittest, PyTest, Data-Driven Testing and Page Object Model

3. Problem Statement

Develop an integrated Python test automation workflow that demonstrates the use of unit-test framework concepts, data-driven testing, PyTest fixtures and assertions, HTML test reporting, and the Page Object Model design pattern. The experiment combines the prescribed M2 practical activities into one cohesive laboratory exercise for organizing, executing, and maintaining automated tests.

4. Objective

To demonstrate practical use of Python unit-testing and automation frameworks by combining data-driven automation, PyTest-based test execution and reporting, fixtures, assertions, and Page Object Model architecture in a single integrated experiment.

5. Tools, Software and Concepts Used

Category	Items
Tools/Software	Python 3; Selenium WebDriver; PyTest; Unittest; VS Code or PyCharm; Web browser; pytest-html
Concepts	Unit testing; test cases; setup and teardown; assertions; test suites; PyTest discovery; fixtures; conftest.py; parameterization/data-driven testing; HTML reporting; Page Object Model; page classes; base class; utility classes; configuration; screenshots on test failure

6. Implementation Steps

1. Set up Python and the required testing and automation packages.
2. Create the basic test structure and define the test cases using an appropriate Python test framework.
3. Organize common setup and teardown operations so that test resources are initialized and cleaned up consistently.
4. Prepare external test data containing multiple valid and invalid test scenarios for data-driven automation.
5. Execute the same test workflow against the available test-data combinations and use assertions to verify the expected behavior.
6. Configure pytest test discovery, test files, test methods, and assertions for automated execution.
7. Create and use pytest fixtures for reusable setup and teardown operations.
8. Use parameterization or data-driven techniques to execute the test with multiple input combinations.
9. Configure pytest HTML reporting so that test execution results can be recorded in a structured report.
10. Restructure the automation into a Page Object Model by separating page locators and UI interaction methods from test logic and assertions.
11. Organize the framework using suitable base, page, test, utility, and configuration components where required.
12. Configure screenshot handling for test failures and maintain a clean, reusable automation structure.

7. Source Code

A. unittest Framework

1. Import the unittest module and create a class derived from unittest.TestCase.
2. Create a first test method whose name starts with test_.
3. Use setUp() for common initialization and tearDown() for cleanup.
4. Use assertion methods such as assertEquals(), assertTrue(), and assertIn().
5. Group test cases using a test suite and execute the suite using a test runner.

Sample unittest implementation:

```
import unittest

class TestCalculator(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print("Class setup executed")

    @classmethod
    def tearDownClass(cls):
```

```

        print("Class teardown executed")

    def test_addition(self):
        result = 2 + 3
        self.assertEqual(result, 5)

    def test_positive_value(self):
        self.assertTrue(10 > 0)

if __name__ == "__main__":
    unittest.main()

```

B. PyTest Framework

6. Install PyTest using `pip install pytest`.
7. Follow naming conventions such as `test_*.py` for test files and `test_*` for test methods.
8. Write test functions using Python assertions.
9. Run all tests, selected files, or selected test methods from the command line.
10. Use fixtures to provide reusable setup and cleanup.
11. Use `conftest.py` to store shared fixtures and configuration.
12. Execute multiple test cases and selected subsets using PyTest options and markers.
13. Move the Selenium test into PyTest by separating test steps and reusable setup.
14. Install `pytest-html` and generate an HTML report after execution.

Sample PyTest fixture and assertion:

```

import pytest

@pytest.fixture
def sample_data():
    return {"username": "test_user", "status": "active"}

def test_user_status(sample_data):
    assert sample_data["status"] == "active"

```

Common commands used:

```

pytest -v

pytest test_login.py -v

pytest -k login -v

pytest --html=report.html --self-contained-html

```

C. Page Object Model Framework

15. Create a `BasePage` class containing common browser actions such as click, type, find, and wait.
16. Create separate page classes for individual application pages.
17. Create test classes that contain test scenarios and assertions.
18. Create utility classes for reusable functions such as CSV reading and screenshot capture.
19. Maintain configuration values such as browser, URL, timeout, and test data separately.

20. Use inheritance so that page classes can reuse BasePage methods.
21. Read test data from a CSV file and pass it to the relevant test.
22. Convert the original Selenium test into a Page Object Model structure.
23. Capture a screenshot automatically when a test fails.

Illustrative BasePage and page class structure:

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver

    def open(self, url):
        self.driver.get(url)

    def click(self, locator):
        self.driver.find_element(*locator).click()

    def type_text(self, locator, value):
        element = self.driver.find_element(*locator)
        element.clear()
        element.send_keys(value)

class LoginPage(BasePage):
    username = ("id", "username")
    password = ("id", "password")
    login_button = ("id", "login")

    def login(self, user, pwd):
        self.type_text(self.username, user)
        self.type_text(self.password, pwd)
        self.click(self.login_button)
```

Note: The locator values and URL must be updated according to the application used in the laboratory. The code snippets are representative examples and should be adjusted to match the actual implementation.

8. Output Screenshots

Figure 1: unittest first test case executed successfully

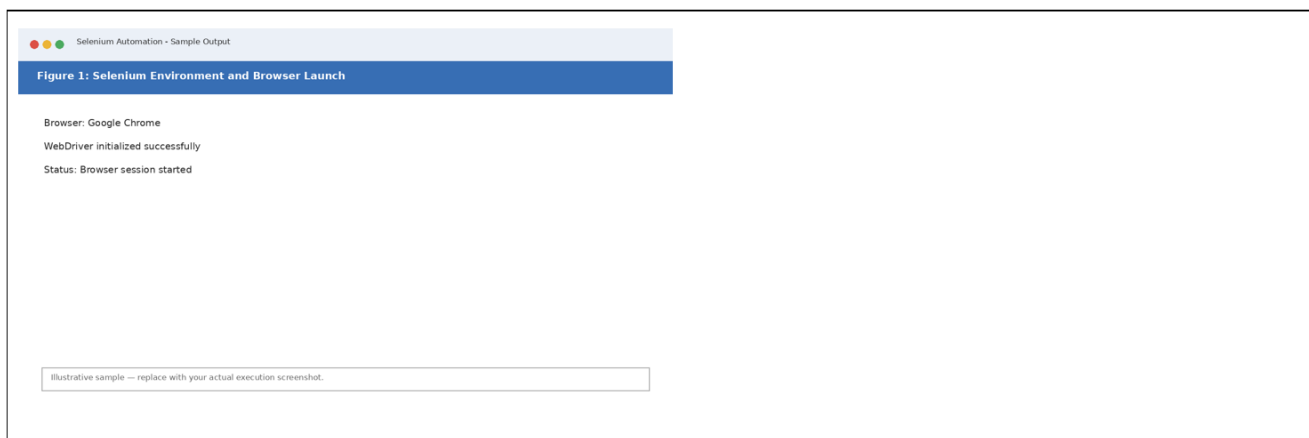


Figure 2: setUp and tearDown methods executed



Figure 3: unittest test suite execution result

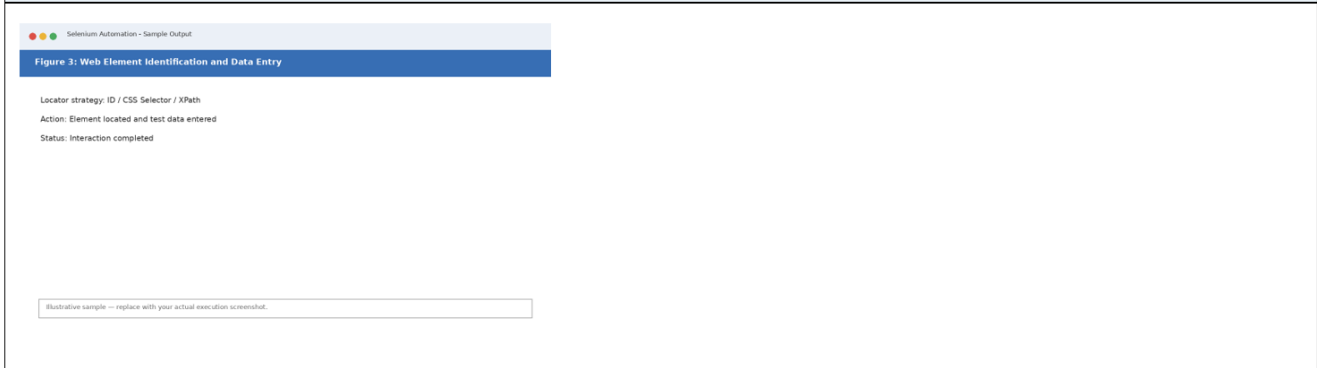


Figure 5: PyTest fixture or conftest.py execution

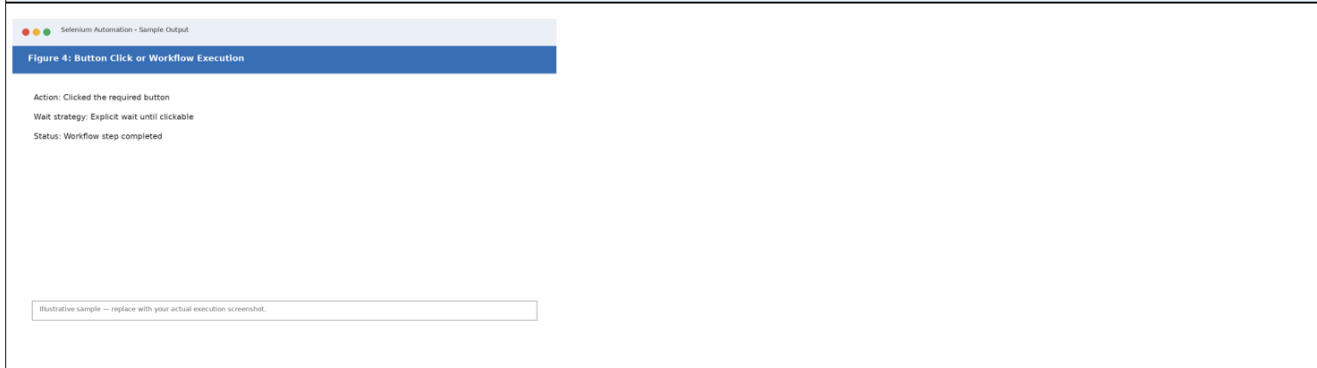
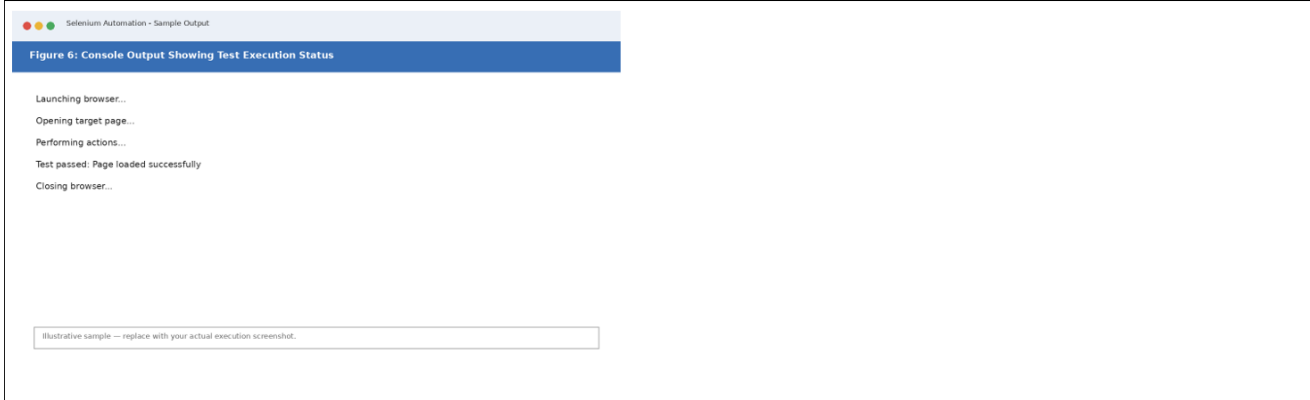


Figure 6: Selenium test executed through PyTest



9. Result

The experiment is designed to demonstrate an integrated unit-testing and automation workflow covering data-driven testing, PyTest execution, fixtures, assertions, HTML reporting, and Page Object Model architecture.

10. Observation

The combined module brings together test organization, reusable setup and teardown, execution with multiple data sets, structured reporting, and separation of test logic from page-specific interaction logic. These practices support maintainable and reusable automated test suites.

11. Conclusion

The experiment provides a consolidated demonstration of the Unit Test Frameworks module by integrating the prescribed data-driven automation, PyTest reporting, and Page Object Model activities into one structured laboratory exercise.